AHB-Based Multi-Master System with Cache and APB Bridge

A

CIAP

for

Advanced Processor Protocols

Submitted by

DEVASHREE SURVE 124BTVL2010(40)

DEPARTMENT OF ELECTRONICS (VLSI DESIGN & TECHNOLOGY)

SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE
(An Autonomous Institute Affilated to University of Mumbai)
MUMBAI - 400 088, MAHARASHTRA (INDIA)

2024

AHB-Based Multi-Master System with Cache and APB Bridge

1.Abstract

This project presents the design and implementation of a simplified AHB-based multi-master system using Verilog RTL. The system consists of two AHB masters, a round-robin arbiter, pipelined data transfer, and multiple slaves including cache, RAM, and an APB peripheral connected through an AHB-to-APB bridge.

The design demonstrates basic bus communication concepts such as arbitration, address decoding, and protocol conversion. A simple cache mechanism is implemented to improve access speed, while the bridge enables communication between AHB and APB.

The system is functionally verified using simulation and RTL schematic visualization. However, the implementation uses a simplified version of the AHB protocol without full compliance features.

2.Introduction

AHB Protocol Overview

The Advanced High-performance Bus (AHB) is a key component of the AMBA (Advanced Microcontroller Bus Architecture) family, widely used in System-on-Chip (SoC) designs to enable high-speed communication between masters, such as processors and controllers, and slaves, including memory and peripheral devices. AHB is designed to support high-performance operations through features such as pipelined data transfers, burst transactions, and multi-master configurations. These capabilities make it suitable for applications requiring efficient and continuous data flow.

The primary signals used in AHB communication include **HADDR** (address bus), **HWDATA** (write data), **HRDATA** (read data), **HWRITE** (read/write control), **HTRANS** (transfer type), **HSEL** (slave select), and **HREADY** (transfer completion signal). These signals collectively manage the control and data flow across the bus.

APB Protocol Overview

The Advanced Peripheral Bus (APB) is a simplified, low-power bus protocol also defined under the AMBA framework. It is primarily used for communication with low-speed peripheral devices such as UARTs, timers, and GPIO modules. Unlike AHB, APB does not support pipelining or burst transfers, which simplifies its design and reduces power consumption.

Key APB signals include **PADDR** (address), **PWDATA** (write data), **PRDATA** (read data), **PWRITE** (read/write control), **PSEL** (slave select), and **PENABLE** (transfer enable). These signals facilitate basic read and write operations with peripheral components.

Project Objective

The objective of this project is to design and implement a simplified System-on-Chip (SoC) communication architecture that demonstrates the interaction between high-speed and low-speed bus protocols.

The system is designed to support multiple masters using arbitration techniques, implement AHB protocol for high-speed data transfer, demonstrate pipelined communication, integrate a basic cache memory for performance enhancement, and establish communication with peripheral devices through an AHB to APB bridge.

3.Block Diagram

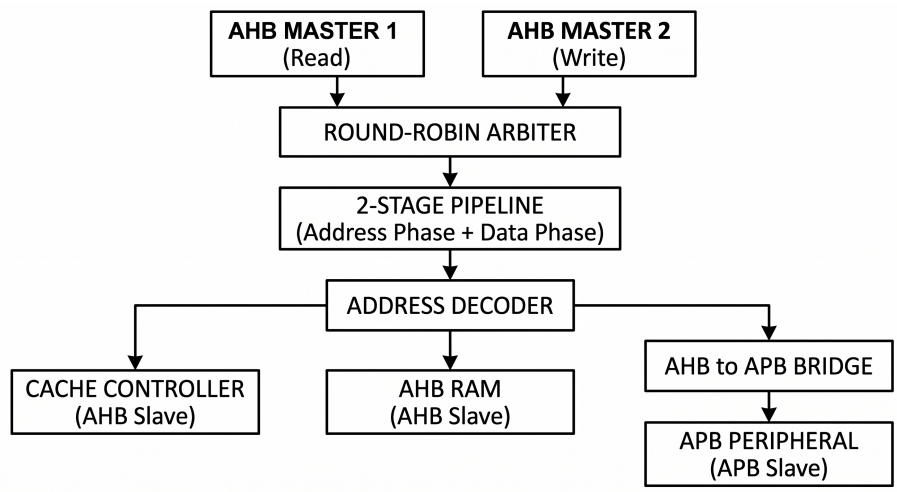

Figure 1: Block diagram of the AHB-based multi-master system with cache, RAM, and APB bridge integration.

Block Diagram Description

The block diagram represents a simplified AHB-based multi-master system designed using Verilog RTL. It illustrates the flow of data and control signals from masters to slaves through different stages of the system.

At the top level, two AHB masters are present:

- **AHB Master 1** – Performs read operations
- **AHB Master 2** – Performs write operations

These masters generate address, data, and control signals required for communication over the AHB bus.

Since both masters may request access to the bus simultaneously, a round-robin arbiter is used to grant access to one master at a time. This ensures fair and controlled usage of the shared bus without conflict.

The selected master's signals are then passed through a 2-stage pipeline, consisting of:

- Address Phase
- Data Phase

This pipelining introduces timing similar to real AHB systems and allows overlapping of operations, thereby improving data transfer efficiency.

Next, the pipelined address is sent to the **address decoder**, which determines the target slave based on the address range. The decoder routes the transaction to one of the following slaves:

- **Cache Controller (AHB Slave)** – Provides fast access to frequently used data using hit/miss logic
- **AHB RAM (AHB Slave)** – Acts as main memory supporting read and write operations
- **AHB to APB Bridge** – Converts AHB protocol signals into APB signals

The AHB to APB bridge further connects to an APB Peripheral (APB Slave), enabling communication with low-speed devices such as UART, timers, or GPIO modules.

Finally, the selected slave processes the request and sends the response (read data or acknowledgment) back through the system, completing the transaction.

Key Features of the Design

- Multi-master AHB architecture
- Round-robin arbitration for fair bus access
- Pipelined data transfer (Address and Data phase)
- Cache integration for performance improvement
- AHB to APB bridging for peripheral communication

4.RTL Implementation

1.AHB Masters

Description:

AHB masters generate address, data, and control signals for read and write operations.

Code Snippet:

Master1

```
initial begin HADDR=0; req=0; end

always @(posedge clk) begin
    req<=1;
    HWRITE<=0;
    HTRANS<=2'b10;

    if(HADDR==3) HADDR<=2;
    else HADDR<=HADDR+1;
end
```

Master2

```
initial begin HADDR=4; req=0; end

always @(posedge clk) begin
    req<=1;
    HWRITE<=1;
    HTRANS<=2'b10;

    HADDR<=HADDR+1;
    HWDATA<=HADDR+8;
end
```

RTL Image:

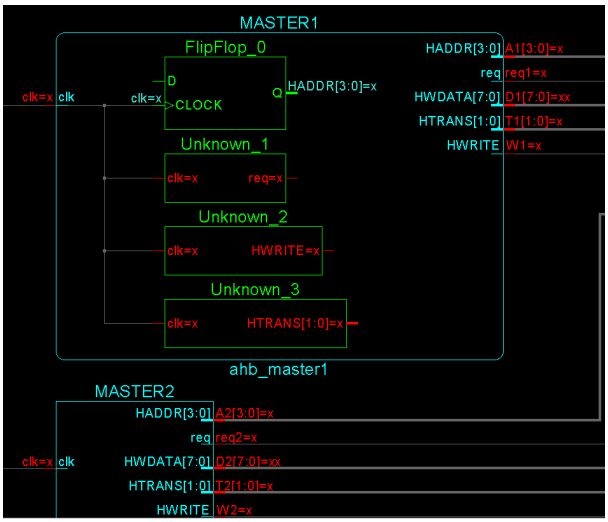

Figure 2: RTL schematic of AHB Master showing generation of address, data, and control signals.

Explanation:

The RTL shows signal generation logic where address and data are updated sequentially and transmitted to the bus.

Arbiter

Description:

Controls bus access between multiple masters using round-robin scheduling.

Code Snippet:

```
always @(*) begin
    if (req1 && req2)
        {grant1, grant2} = turn ? 2'b10 : 2'b01;
    else if (req1)
        {grant1, grant2} = 2'b10;
    else if (req2)
        {grant1, grant2} = 2'b01;
    else
        {grant1, grant2} = 2'b00;
end
```

Explanation:

The arbiter RTL shows the selection logic that determines which master is granted control of the bus at a given time.

RTL Image:

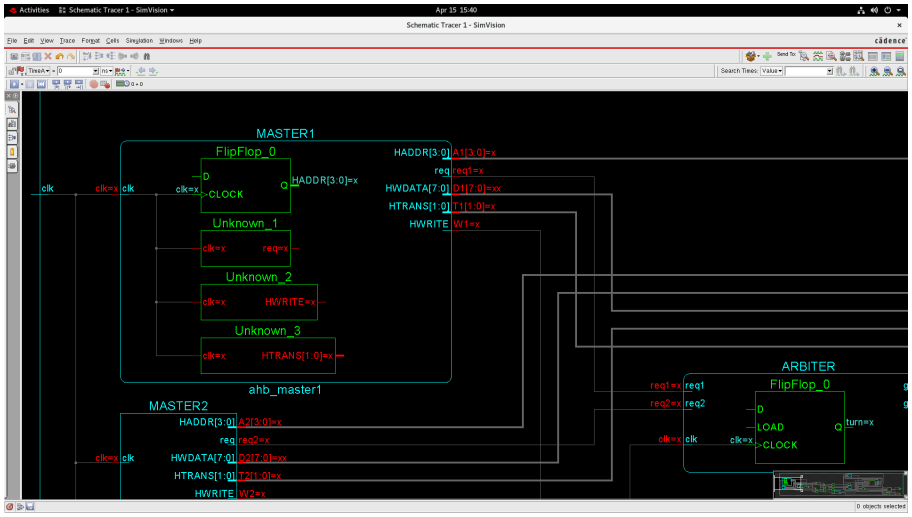

Figure 3: RTL schematic of round-robin arbiter controlling bus access between multiple AHB masters.

Pipeline

Description:

Implements a 2-stage pipeline consisting of address and data phases.

Code Snippet:

```
always @(posedge clk) begin
    A_s1<=HADDR; D_s1<=HWDATA; W_s1<=HWRITE; T_s1<=HTRANS;
    A_s2<=A_s1;  D_s2<=D_s1;  W_s2<=W_s1;  T_s2<=T_s1;
end

ahb_master1 MASTER1(clk,req1,A1,W1,D1,T1);
ahb_master2 MASTER2(clk,req2,A2,W2,D2,T2);
ahb_arbiter ARBITER(clk,req1,req2,g1,g2);
```

Figure 4: 2-stage pipeline showing address and data phase registers.

Explanation:

The design implements a simplified multi-master AHB system consisting of two masters, an arbiter, and a 2-stage pipeline. Each master independently generates request signals (`req1`, `req2`) along with address, data, and control signals for bus transactions.

The arbiter receives these request signals and generates grant signals (`g1`, `g2`) to ensure that only one master accesses the shared bus at any given time. Based on the grant signal, a multiplexer selects the corresponding master's signals (address, data, and control) and forwards them to the bus.

To enhance performance, a 2-stage pipeline is implemented:

- **Stage 1 (Address Phase):** Captures the selected master's address and control signals
- **Stage 2 (Data Phase):** Delays the signals by one clock cycle to process data transfer

Flip-flops in the RTL represent these pipeline stages, introducing controlled delay and enabling overlapping of consecutive transactions. This pipelined approach follows the AHB protocol and improves throughput by reducing idle cycles.

Key Features:

- Two AHB masters generating independent requests
- Arbiter-controlled bus access using grant signals (`g1`, `g2`)
- Multiplexer-based master selection
- 2-stage pipelining for improved efficiency
- Support for high-speed pipelined data transfer

Address Decoder

Description:

Selects the appropriate slave based on the address range.

Code Snippet:

```
always @(*) begin
    sel_cache = 0;
    sel_ram   = 0;
    sel_apb   = 0;

    if (HADDR < 4)
        sel_cache = 1;
    else if (HADDR < 8)
        sel_ram = 1;
    else
        sel_apb = 1;
end
```

RTL Image:

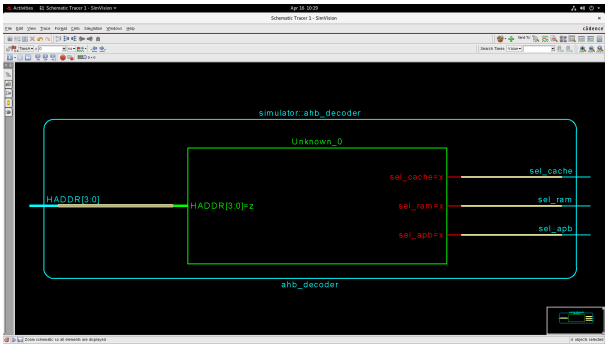

Figure 5: RTL schematic of address decoder selecting cache, RAM, or APB bridge based on address range.

Explanation:

The decoder routes transactions to cache, RAM, or bridge depending on the address value.

Cache Controller

Description:

Stores frequently accessed data and determines hit or miss conditions.

Code Snippet:

```
if (HSEL && HTRANS==2'b10) begin

    if (HWRITE) begin
        cache_data[index] <= HWDATA;
        tag[index] <= tag_in;
        valid[index] <= 1;
        HRDATA <= HWDATA;
        hit <= 1;
    end
    else begin
        if (valid[index] && tag[index]==tag_in) begin
            HRDATA <= cache_data[index];
            hit <= 1;
        end else begin
            // FIXED REFILL
            cache_data[index] <= RAM_DATA;
            tag[index] <= tag_in;
            valid[index] <= 1;
            HRDATA <= RAM_DATA;
            hit <= 0;
        end
```

RTL Image:

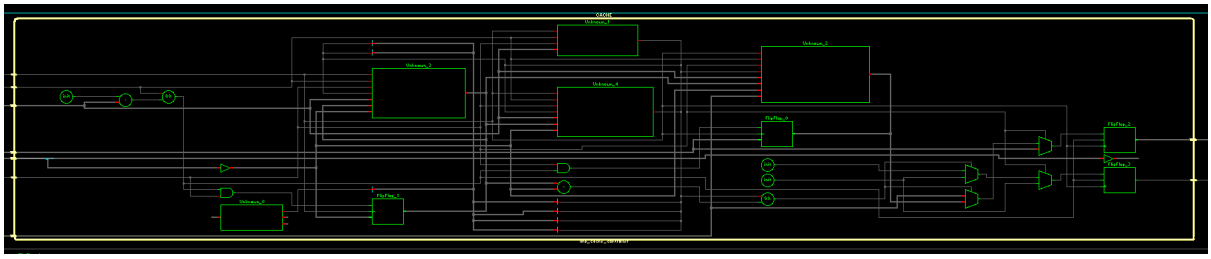

Figure 6: RTL schematic of cache controller showing storage elements and hit/miss decision logic.

Explanation:

The RTL shows memory elements for storing cache lines along with comparison logic for tag matching.

RAM

Description:

Acts as the main memory for storing and retrieving data.

Code Snippet:

```
reg [7:0] memory [0:15];
integer i;

initial begin
    for(i=0;i<16;i=i+1)
        memory[i] = i;
end

always @(posedge clk) begin
    if (HSEL) begin
        if (HWRITE)
            memory[HADDR] <= HWDATA;
        else
            HRDATA <= memory[HADDR];
    end
```

RTL Image:

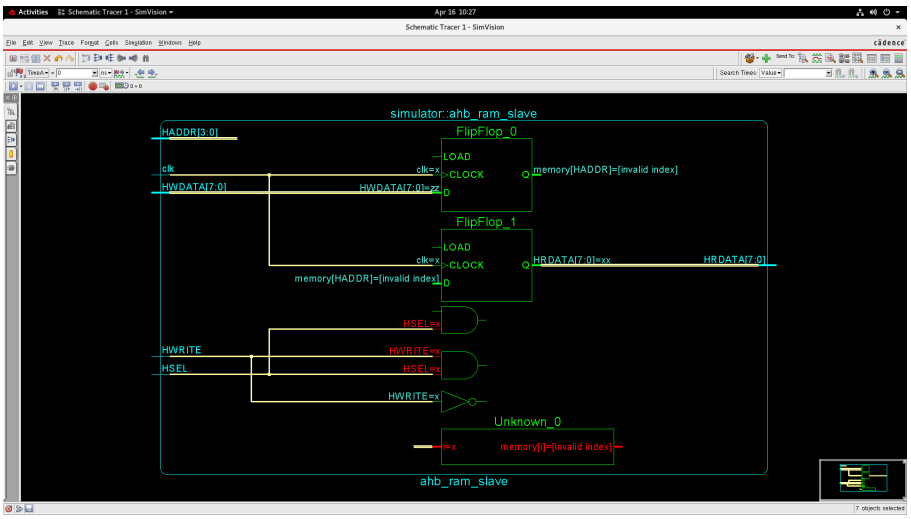

Figure 7: RTL schematic of AHB RAM showing read and write memory operations.

Explanation:

The memory array is used to store data during write operations and retrieve data during read operations.

AHB to APB Bridge

Description:

Converts AHB protocol signals into APB protocol signals for peripheral communication.

Code Snippet:

```
reg state;

initial state = 0;

always @(posedge clk) begin
    case(state)

    0: begin
        if (HSEL && HTRANS==2'b10) begin
            PADDR  <= HADDR;
            PWRITE <= HWRITE;
            PWDATA <= HWDATA;
            PSEL   <= 1;
            PENABLE<= 0;
            state  <= 1;
        end
    end

    1: begin
        PENABLE <= 1;

        if (!HWRITE)
            HRDATA <= PRDATA;

        PSEL <= 0;
        state <= 0;
    end
```

RTL Image:

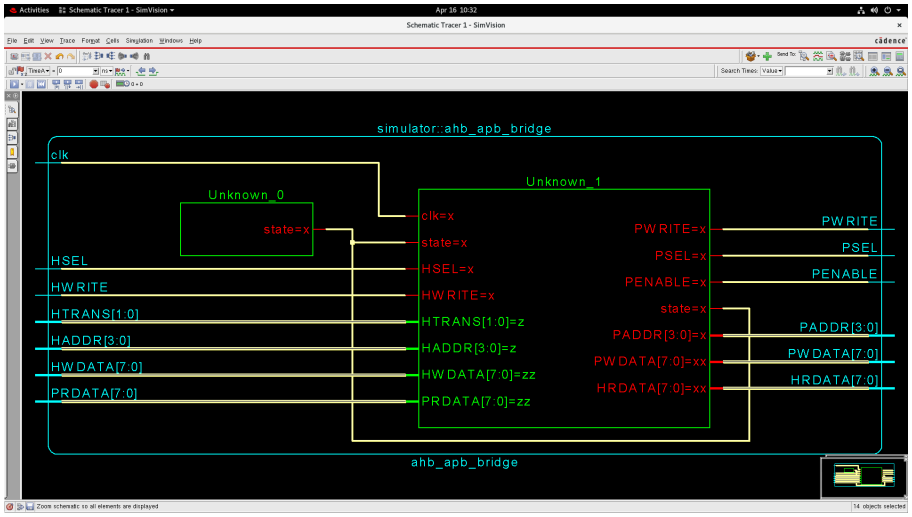

Figure 8: RTL schematic of AHB to APB bridge illustrating protocol conversion and signal mapping.

Explanation:

The RTL demonstrates mapping of AHB signals to APB signals and control logic required for protocol conversion.

APB Slave Device

Description:

The APB slave device acts as a low-speed peripheral connected through the AHB-to-APB bridge. It performs read and write operations based on APB control signals. When both PSEL and PENABLE are active, the module either writes data into memory (if PWRITE = 1) or reads data from memory (if PWRITE = 0). The memory is initialized with predefined values for simulation.

Code Snippet:

```
reg [7:0] memory [0:15];
integer i;

initial begin
    for(i=0;i<16;i=i+1)
        memory[i] = i + 20;
end

always @(posedge clk) begin
    if (PSEL && PENABLE) begin
        if (PWRITE)
            memory[PADDR] <= PWDATA;
        else
            PRDATA <= memory[PADDR];
    end
```

RTL Image:

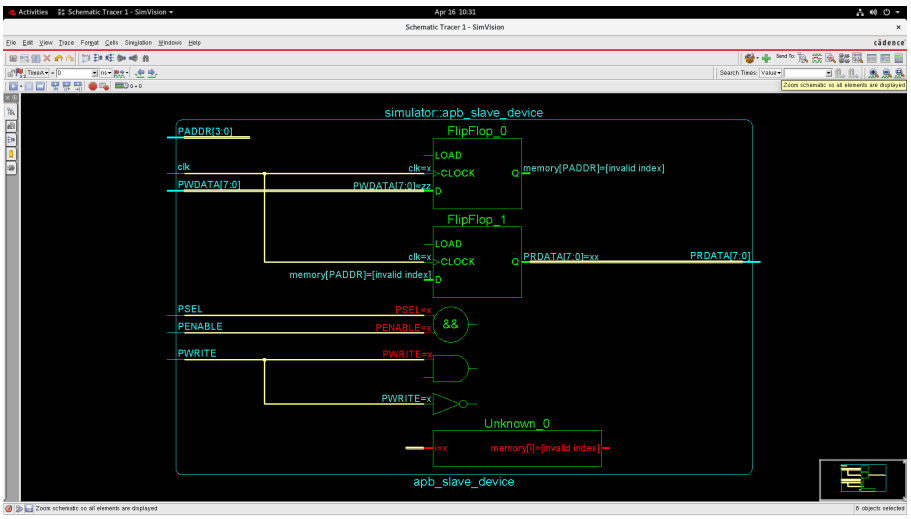

Figure 9: RTL schematic of APB slave device showing memory array and read/write control using PSEL, PENABLE, and PWRITE signals.

Top Module

Description:

The top module integrates all components of the system, including AHB masters, arbiter, pipeline stages, address decoder, cache controller, RAM, and the AHB-to-APB bridge with APB slave. It manages the overall data flow from masters to slaves.

The arbiter selects one master, whose signals are passed through a 2-stage pipeline. The pipelined address is decoded to select the appropriate slave (cache, RAM, or APB). Data from the selected slave is then routed back through a multiplexer to complete the transaction.

RTL Image:

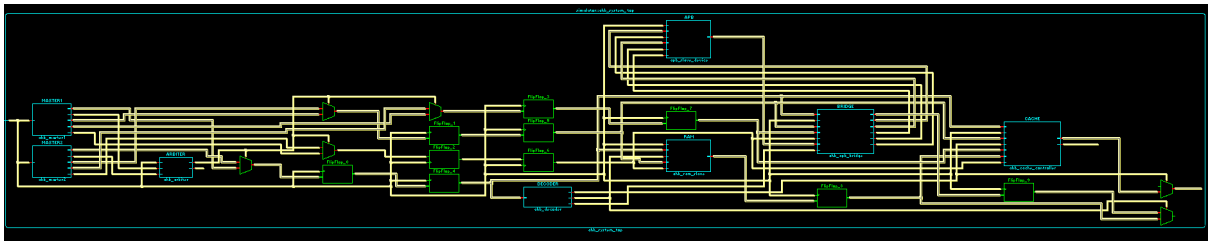

Figure 10: RTL schematic of top module showing integration of masters, arbiter, pipeline, decoder, cache, RAM, and AHB to APB bridge.

5.Functional Validation & Simulation Analysis

The system was simulated using Cadence Xcelium (`xrun`) with a testbench generating clock signals and monitoring address and data outputs. The simulation verifies correct interaction between masters, arbiter, pipeline, decoder, cache, RAM, and the APB bridge.

Observed Simulation Output

```
T=0 ADDR= x DATA=  x
T=15 ADDR= 4 DATA=  x
T=25 ADDR= 5 DATA=  4
T=35 ADDR= 2 DATA=  x
T=45 ADDR= 7 DATA=  4
T=55 ADDR= 2 DATA=255
T=65 ADDR= 9 DATA=  x
T=75 ADDR= 2 DATA=255
T=85 ADDR=11 DATA=  x
T=95 ADDR= 2 DATA=255
T=105 ADDR=13 DATA=  x
T=115 ADDR= 2 DATA=255
T=125 ADDR=15 DATA=  x
T=135 ADDR= 2 DATA=255
T=145 ADDR= 1 DATA=255
T=155 ADDR= 2 DATA=  8
T=165 ADDR= 3 DATA=255
T=175 ADDR= 2 DATA= 10
T=185 ADDR= 5 DATA=  4
T=195 ADDR= 2 DATA=255
```

Key Simulation Behavior

T = 0 → Initial State

Address and data show unknown values (`x`). This occurs because registers are not initialized and pipeline stages are empty. This behavior is expected in RTL simulation.

T = 25 → RAM Read

ADDR = 5 results in DATA = 4. Address 5 lies in the RAM range (4–7), where memory is initialized as `memory[i] = i`. The expected value is 5, but the output is 4 due to pipeline delay. The data corresponds to the previous address (ADDR = 4).

T = 45 → RAM Access

ADDR = 7 results in DATA = 4, again showing delayed response. This confirms the effect of pipelining.

T = 55 → Cache Access

ADDR = 2 results in DATA = 255. This address falls in the cache region. The value differs from RAM, indicating that the cache has stored data from a previous operation. This confirms cache functionality, although timing is not perfectly controlled.

T = 65–125 → APB Region

Addresses such as 9, 11, and 13 produce DATA = x. These correspond to APB accesses. The unknown output occurs because the APB bridge introduces additional delay and the

data is not yet available when sampled. This confirms that the bridge is functional but timing is simplified.

T = 155 → Updated Data
ADDR = 2 results in DATA = 8. This reflects updated memory content due to write operations (`HWDATA <= HADDR + 8`). The cache later reflects this updated value.

T = 175 → Further Update
ADDR = 2 results in DATA = 10. This shows continuous updates from the write master, confirming correct write path and data propagation.

6.Design Evaluation & Trade-offs

Successfully Implemented Features

The proposed design successfully implements a multi-master AHB system with key architectural components. It includes a round-robin arbitration mechanism to manage bus access among multiple masters, along with accurate address decoding logic to select appropriate slaves. A 2-stage pipeline is incorporated to improve data transfer efficiency, while a basic cache controller supports hit and miss operations. Additionally, the system includes RAM read/write functionality and integrates an AHB to APB bridge to enable communication with low-speed peripherals.

Simplifications and Limitations

Despite achieving the core functionality, the design includes several simplifications. The AHB protocol is not fully implemented, as it lacks support for burst transfers, `HRESP` response handling, and complete `HREADY` control. The cache design is also simplified, with no implementation of replacement policies such as LRU or FIFO, and its timing behavior is not fully optimized. Similarly, the APB interface uses simplified timing, resulting in data that is not always properly synchronized. Furthermore, the overall timing across modules is not perfectly aligned, as the pipeline stages are not fully synchronized with all system components.

Design Trade-offs

Design Choice	Advantage	Limitation
Simple AHB model	Easy to implement	Not protocol-complete
Direct-mapped cache	Low complexity	Limited efficiency
Basic bridge FSM	Functional	No timing optimization
2-stage pipeline	Demonstrates concept	Causes delay mismatch

7. Conclusion

The project successfully demonstrates a simplified AHB-based multi-master system using Verilog RTL, integrating key components such as arbitration, pipelining, cache, RAM, and an AHB-to-APB bridge. The system exhibits correct functional behavior for basic data transfer and interaction between modules.

However, due to simplified timing and partial protocol implementation, certain inconsistencies such as delayed responses and undefined values (`x`) are observed in the simulation. These limitations highlight the importance of precise timing control and complete protocol adherence in real-world designs.

Future improvements can include implementing full AHB protocol features like HREADY, HRESP, and burst transfers, enhancing the cache with replacement policies such as LRU or FIFO, improving synchronization of pipeline timing across modules, and optimizing the APB bridge for better timing accuracy. The design can also be extended by adding more masters and slaves or upgrading to advanced protocols like AXI for higher performance systems.